

1 Project Overview

Problem: Pakistan's 2,000+ legal codes are locked inside dense government PDFs, inaccessible to ordinary citizens. Legal consultations cost thousands of rupees and are unavailable in rural areas.

Legal Code Assistant is an AI-powered SaaS web application that answers legal questions in plain English, Urdu, and Roman Urdu — for free. It uses Retrieval-Augmented Generation (RAG) on 2,464 official Pakistani law PDFs to provide instant, cited answers for citizens, police officers, and lawyers.

2 Features Being Built

■ AI Legal Chatbot Ask any question about Pakistani law in English/Urdu/Roman Urdu. Cites exact PPC/CrPC sections.	■ FIR Generator Describe an incident → AI auto-generates a legally correct FIR. References CrPC Section 154.
■ Voice Assistant Speak your legal question in any language. Speech-to-text → RAG query → spoken answer.	■ Case Outcome Predictor Input case facts → AI predicts bail probability and conviction likelihood.
■ Document Analyzer Upload FIR or contract → AI highlights key clauses, risks, and applicable laws.	■ Self-Assessment Tool Instantly check penalty and bail status for any offense under Pakistani law.

3 Progress Summary

Step 1 — Project Setup & Monorepo	100%
Step 3 — Data Collection (2,464 PDFs)	100%
Step 8 — Frontend UI (All Pages)	100%
Step 2 — Account Creation	30%
Steps 4–5 — PDF Extraction & Embeddings	0%
Steps 6–10 — RAG API & Backend	0%
Steps 11–18 — Advanced Features	0%
Steps 20–30 — Testing, Deploy, FYP Docs	0%

Step 1 **Project Setup & Monorepo Structure**

✓ DONE

- Next.js 16 + React 19 + TypeScript initialized
- Tailwind CSS v4 + Shadcn/UI (base-nova) configured
- Monorepo: frontend/, ai-ml-module/, legal-data/, backend-core/
- npm packages: Zustand, TanStack Query, Framer Motion, Lucide React
- Git repository with proper .gitignore

Step 3 **Data Collection — 2,464 Pakistani Law PDFs**

✓ DONE

- Custom Selenium scraper:
ai-ml-module/scripts/download_pakistan_code_pdfs.py
- Source: pakistancode.gov.pk — official Estacode (all 17 categories)
- 2,464 PDF files — Pakistani laws from 1840 to 2021
- Stored in legal-data/raw-pdfs/

Category	Count	Category	Count
Criminal Laws	449	Police Laws	212
Civil Laws	401	Companies Laws	187
Family Laws	300	Land/Property Laws	118
Labour Laws	227	Islamic/Religious Laws	108
Service Laws	219	Banking/Financial Laws	74
Total			2,464

Step 8 **Frontend UI — Full Next.js App Router Migration**

✓ DONE

- Custom Navy + Gold design system, Playfair Display + Inter fonts
- Landing page: hero, voice search bar, role cards, features section, CTA
- Auth pages: Login (split-panel) + Signup
- Pricing page: Free / Pro (Rs.999/mo) / Enterprise tiers
- Chat system: sidebar layout, /chat role selector, /chat/[chatId] dynamic route
- Assessment tool: penalty + bail checker (6 offense categories)
- Settings: dark mode, language selector (English / Urdu / Roman Urdu)
- Profile: user info, plan status, Pro upgrade, sign out
- State: Zustand chatStore.ts — chat history + auth state
- Voice input: Web Speech API hook — real-time speech-to-text
- Build: TypeScript passing, 0 errors, 11 routes generating (8 static + 1 dynamic)

Step 2

Account Creation (Groq, Supabase, Qdrant, Vercel)

■ IN
PROGRESS

All service accounts being set up. All have free tiers — zero cost for FYP scope.

Step 4 — PDF Text Extraction [NEXT]

Use `pdfplumber` (Python, free) to extract text from all 2,464 PDFs. Parse into structured JSON with section numbers, titles, and content. Expected: 50,000–200,000 law sections. Output to `legal-data/extracted/`.

Step 5 — Vector Embeddings + Qdrant Upload

Convert each section into 384-dim vectors using `paraphrase-multilingual-MiniLM-L12-v2` (supports Urdu + English, runs on CPU, completely free). Upload to Qdrant cloud free tier.

Step 6 — Semantic Search Function

Next.js API route: user query → embed → Qdrant nearest-neighbour search → return top-K matching law sections with metadata (category, year, section number).

Step 7 — RAG Chat API (Groq Llama 3.1-70B)

Core AI endpoint: retrieved sections + user question → Groq API → answer with citations. Connect to existing chat UI (replacing current mock responses).

Step 9 — Authentication (Supabase Auth)

Real login/signup via Supabase Auth. Session management, protected routes, user profiles in PostgreSQL.

Step 10 — Usage Limits & Tracking

Free: 10 queries/day. Pro: unlimited. Track in Supabase, enforce in Next.js middleware.

Step 11 — FIR Generator

Multi-step form → Groq generates properly formatted FIR → PDF export. References CrPC Section 154.

Step 12 — Voice Assistant (Full Integration)

Voice input already built on frontend. Connect to RAG backend. Add TTS output for AI responses.

Steps 13–14 — Case Predictor & Document Analyzer

Case predictor: facts → probability analysis. Document analyzer: upload FIR/contract → AI extracts risks.

Steps 15–16 — Payment System (JazzCash)

JazzCash Business API for Rs. 999/month Pro subscriptions. Webhook handling for subscription status.

Steps 17–18 — User Dashboard & Query History

Dashboard: usage stats, saved queries, subscription. Full query history with search and export.

Steps 20–24 — Performance, Testing & Bug Fixes

Optimization, comprehensive error handling, SEO meta tags, PWA support, end-to-end testing.

Step 25 — Production Deployment

Frontend → Vercel. Database → Supabase Cloud. Configure custom domain, SSL, environment variables.

Steps 26–27 — Launch & User Feedback

Beta launch, target 100+ users. Iterate. Revenue target: Rs. 99,900/month from 100 Pro users.

Steps 28–30 — FYP Documentation & Submission

Full FYP report, system documentation, demo preparation, final submission and demo day.

6 Technology Stack

Zero-Cost Architecture: Every tool has a free tier sufficient for FYP scope. No credit card required to build and launch the entire system.

Layer	Technology	Purpose	Cost
Frontend	Next.js 16 + React 19 + TypeScript	Web app, SSR/SSG, API Routes	Free
	Tailwind CSS v4 + Shadcn/UI	Styling, reusable components	Free
	Zustand + TanStack Query	State management, data fetching	Free
AI / LLM	Groq — Llama 3.1-70B	Answer generation with legal citations	Free tier
Embeddings	multilingual-MiniLM-L12-v2	Text → 384-dim vectors (Urdu+English)	Free (local)
Vector DB	Qdrant Cloud	Semantic search over 50K-200K sections	Free 1GB
Database	Supabase (PostgreSQL)	Users, auth, subscriptions, history	Free tier
Python	pdfplumber + sentence-transformers	PDF extraction → embeddings pipeline	Free
Payments	JazzCash Business API	Rs.999/month Pro subscriptions	% per txn
Deploy	Vercel	Frontend hosting, serverless functions	Free tier
Data	pakistancode.gov.pk (Estacode)	2,464 official PDFs (1840–2021)	Public

7 RAG Architecture



uning?

Fine-tuning a large language model requires expensive GPU time (thousands of dollars) and a lot of data. RAG achieves the same result at zero cost — the model stays general and retrieves relevant law text at query time and pass it as context. This is exactly how models like ChatGPT with plugins operate.